

Le protocole d'extension et les serveurs essentiels

Qu'est-ce que MCP ?

MCP (Model Context Protocol) est le standard ouvert qui permet à Claude Code de se connecter à des outils externes via un protocole JSON-RPC 2.0. Chaque MCP server expose des outils supplémentaires, nommés selon la convention `mcp_<server>_<tool>`. Claude les utilise exactement comme ses outils natifs, avec le même système de permissions.

Architecture de base : Claude Code (client) communique avec le MCP server via stdio ou HTTP. Le server démarre au premier usage et reste actif pendant toute la session.

Serveurs les plus utilisés

Serveur	Usage principal
Context7	Documentation officielle de bibliothèques
Sequential Thinking	Raisonnement multi-étapes structuré
Grepai	Recherche sémantique + graphe d'appels
Serena	Navigation symbolique, refactoring
Playwright	Automatisation browser, tests E2E
GitHub	Issues, PRs, repositories

Configuration dans settings.json

Les MCP servers se déclarent dans `~/.claude/settings.json` (global) ou `.claude/settings.json` (projet).

```
{  "mcpServers": {    "context7": {      "command": "npx",      "args": [        "-y",        "@upstash/context7-mcp"      ],      "env": {}    },    "github": {      "command": "npx",      "args": [        "@github/mcp-server"      ],      "env": {        "GITHUB_TOKEN": "${GITHUB_TOKEN}"      }    }  } }
```

Transport : local (stdio) vs distant (HTTP)

stdio (local) : le server tourne en tant que processus enfant sur la machine. Aucune exposition réseau, idéal pour les outils locaux comme grepai (Ollama) ou Serena.

HTTP/SSE (distant) : le server est accessible via une URL. Utilisé pour les services hébergés comme Figma MCP (`https://mcp.figma.com/mcp`) ou des serveurs d'équipe partagés.

```
{  "mcpServers": {    "figma": {      "transport": "http",      "url": "https://mcp.figma.com/mcp"    }  } }
```

MCP Apps (SEP-1865)

Extension stable depuis janvier 2026, co-développée par Anthropic et OpenAI. Elle permet aux MCP servers de retourner des interfaces interactives (HTML/JS) en plus des réponses texte classiques, rendues dans un iframe sandboxé côté client.

Cas d'usage : dashboards de données, formulaires complexes, visualisations temps réel directement dans la conversation.

alwaysLoad (v2.1.121)

Par défaut, les outils MCP sont chargés en différé (via `ToolSearch`). Pour les serveurs avec 1 à 5 outils critiques

utilisés à chaque session, utilisez `alwaysLoad: true` pour les rendre toujours disponibles sans appel de recherche préalable :

```
{  "mcpServers": {    "mon-serveur-critique": {      "command": "npx",      "args": [        "mon-mcp-server"      ],      "alwaysLoad": true    }  } }
```

Éviter sur les serveurs avec de nombreux outils (20+) — le coût tokens s'accumule.

Limites importantes

Un MCP server n'accède pas à l'historique de la conversation, il ne voit que les paramètres passés lors de l'appel. Chaque appel est indépendant sauf si le server implémente lui-même un cache. Il ne peut pas non plus modifier le system prompt de Claude ni bypasser le système de permissions.

Coût tokens : chaque server chargé ajoute environ 2K tokens d'overhead par session (définitions des outils). Charger uniquement ce qui est nécessaire pour le projet en cours.